ChartNav — Workflow State Machine Build

Phase: strict encounter state machine + filtered listing
Repo: `thekidd2227/chartnav-platform` · Generated 2026-04-17

Contents

1. Executive summary
2. 01 — Current state
3. 02 — Workflow state machine
4. 03 — API endpoints
5. 04 — Data model
6. 05 — Build log
7. 06 — Known gaps + verification matrix
8. 07 — Auth & org scoping
9. Diagram — System architecture
10. Diagram — Encounter status machine
11. Diagram — ER
12. Diagram — API / data flow

Executive summary

This document consolidates **all** ChartNav backend phases delivered to date. It covers the workflow spine, the strict encounter state machine, encounter filtering, and the dev auth + org scoping layer that just landed.

Phase 1 — Workflow spine. Alembic migration `a1b2c3d4e5f6` introduced `encounters` and `workflow_events` tables with FKs and indexes. Read and write endpoints for both. Idempotent seed.

Phase 2 — State machine + filtering. `POST /encounters/{id}/status` enforces explicit forward + rework edges; everything else returns `400 invalid_transition` with an allowed-next-states listing. `GET /encounters` accepts `organization_id`, `location_id`, `status`, `provider_name` filters.

Phase 3 — Dev auth + org scoping (this phase). New `apps/api/app/auth.py` resolves the caller from `X-User-Email`, looks up the user, and attaches a `Caller` dataclass with authoritative `organization_id`. Every encounter route is now org-scoped: cross-org reads return 404, cross-org

body/query assertions return 403. Seed now provisions two tenants (`demo-eye-clinic` , `northside-retina`) so scoping is actually exercised.

Preserved untouched: `/health` , `/` , Alembic history, SQLite dev workflow, the entire state-machine + filtering surface, and the existing workflow_events model.

01 — Current state

ChartNav — Current State

As of: 2026-04-17 (phase: dev auth + org scoping)

Repo layout (relevant)

```
chartnav-platform/
├── apps/
│   ├── api/
│   │   ├── app/main.py              # app factory, CORS, router mount
│   │   ├── app/auth.py              # NEW — dev auth + org-scoping helpers
│   │   ├── app/api/routes.py        # HTTP handlers, all encounter routes org-scoped
│   │   ├── alembic.ini
│   │   ├── alembic/env.py
│   │   ├── alembic/versions/        # 2 migrations (unchanged this phase)
│   │   ├── scripts_seed.py          # now seeds 2 orgs for scoping proof
│   │   └── pyproject.toml
│   └── web/                         # untouched this phase
├── infra/docker/docker-compose.yml
└── docs/
    ├── build/                       # living docs
    ├── diagrams/                    # Mermaid sources
    ├── final/                       # consolidated HTML/PDF (regenerated)
    └── releases/
```

Runtime baseline

- Python 3.11+, FastAPI, raw `sqlite3` driver.
- SQLite at `apps/api/chartnav.db` (gitignored).
- Alembic head: `a1b2c3d4e5f6` (no new migrations this phase).
- CORS still wide-open (`*`) for local dev.
- Dev auth: every protected route reads `X-User-Email` and resolves the

caller from the `users` table. See `07-auth-and-scoping.md` .

Verified working endpoints

Open (no auth)

- `GET /health`, `GET /`
- `GET /organizations`, `GET /locations`, `GET /users`

— intentionally left open this phase; see known gaps.

Authenticated (require `X-User-Email`)

- `GET /me` — NEW. Returns the resolved caller context.
- `GET /encounters` — caller's org only; rejects cross-org filter.
- `GET /encounters/{id}` — 404 if cross-org.
- `GET /encounters/{id}/events` — 404 if cross-org.
- `POST /encounters` — forces `organization_id` to caller's org; location must also belong.
- `POST /encounters/{id}/events` — 404 if cross-org.
- `POST /encounters/{id}/status` — 404 if cross-org; strict state machine still applies.

Seeded tenants

org_id	slug	admin email	encounters
1	`demo-eye-clinic`	`admin@chartnav.local`	PT-1001, PT-1002
2	`northside-retina`	`admin@northside.local`	PT-2001

02 — Workflow state machine

Encounter Workflow State Machine

Status values

Status	Meaning
`scheduled`	Visit is booked, patient has not arrived.
`in_progress`	Provider is seeing the patient / actively charting.
`draft_ready`	Note draft has been generated and is awaiting review.
`review_needed`	Reviewer flagged the note — needs revision or sign-off.
`completed`	Note signed off; encounter closed. **Terminal.**

Allowed transitions

Forward flow:

```
scheduled → in_progress → draft_ready → review_needed → completed
```

Explicit rework transitions (documented + enforced):

From	To	Purpose
`draft_ready`	`in_progress`	Draft rejected, return to charting.
`review_needed`	`draft_ready`	Reviewer kicks note back for rewrite.

Rejected transitions (examples that return HTTP 400)

- `scheduled → completed`
- `scheduled → review_needed`
- `scheduled → draft_ready`
- `in_progress → scheduled`
- `in_progress → completed` (must pass through draft_ready + review_needed)

- `completed → anything` (terminal)
- Any unknown status string (`invalid_status`)

Error body format:

```
{"detail":"invalid_transition: in_progress -> completed is not permitted; allowed next s
```

Idempotency

Posting the **same** status the encounter is already in is a no-op: the row

is returned unchanged and **no** `status_changed` event is appended.

Side effects on successful transition

Transition target	Timestamp action
`in_progress`	set `started_at = now()` if currently NULL
`completed`	set `completed_at = now()` ; also set `started_at = now()` if it was NULL
other	no timestamp mutation

Every successful transition appends a `workflow_events` row:

```
{
  "event_type": "status_changed",
  "event_data": {"old_status": "...", "new_status": "..."}
}
```

Diagram

See `docs/diagrams/encounter-status-machine.md` (Mermaid).

Source of truth

`apps/api/app/api/routes.py` :

- `ALLOWED_STATUSES` — the 5 permitted status strings.

- `ALLOWED_TRANSITIONS` — `dict[str, set[str]]`, one entry per source state.
- `update_encounter_status()` — the single enforcement point.

03 — API endpoints

API Endpoints

Base URL (local dev): `http://127.0.0.1:8000`.

Auth: every endpoint tagged 🔒 requires header `X-User-Email: <seeded user email>`.

See `07-auth-and-scoping.md` for the full model.

System (open)

Method	Path	Response
GET	`/health`	`{"status":"ok"}`
GET	`/`	`{"service":"chartnav-api","version":"0.1.0"}`

Identity

Method	Path	Auth	Response
GET	`/me`	🔒	`{user_id, email, full_name, role, organization_id}`

Org / location / user (open, read-only)

These are **not** org-scoped this phase. Tracked in `06-known-gaps.md`.

Method	Path
GET	`/organizations`
GET	`/locations`
GET	`/users`

Encounters (🔒 org-scoped)

GET `/encounters`

Always filtered to `caller.organization_id`.

Additional optional query parameters (all AND-ed, parameterized):

Param	Type	Notes
`organization_id`	int≥1	Must equal `caller.organization_id` — else 403.
`location_id`	int≥1	Narrows within caller org.
`status`	string	Must be in `ALLOWED_STATUSES` or 400.
`provider_name`	string	Exact match.

GET `/encounters/{encounter_id}`

Returns encounter if it belongs to caller's org. Otherwise **404** **`encounter_not_found`** — same response whether it doesn't exist or belongs to another org (no cross-org existence oracle).

GET `/encounters/{encounter_id}/events`

404 if the parent encounter doesn't exist or is cross-org.

Returns events oldest-first; `event_data` is JSON-parsed when valid.

POST `/encounters`

Body:

```
{
  "organization_id": 1,
  "location_id": 1,
  "patient_identifier": "PT-1003",
  "patient_name": "Alex Chen",
  "provider_name": "Dr. Ortiz",
  "scheduled_at": "2026-04-18T14:30:00Z",
  "status": "scheduled"
}
```

Rules:

- `organization_id` **must equal** `caller.organization_id` → else 403 `cross_org_access_forbidden`.
- `location_id` must exist and belong to caller's org → else 400 `location_not_found` or 403.
- `status` defaults to `"scheduled"`; only `scheduled` or `in_progress` permitted at creation (deeper states can't be forged).
- Appends `workflow_events` row `encounter_created` with `{status, created_by}`.

POST `/encounters/{encounter_id}/events`

404 if cross-org. Body:

```
{ "event_type": "note_draft_completed", "event_data": { "template": "glaucoma-initial" }
```

`event_data` may be any JSON value. Objects/arrays stored as canonical JSON.

POST `/encounters/{encounter_id}/status`

404 if cross-org. Body: `{ "status": "draft_ready" }`.

- Validated against `ALLOWED_STATUSES` and `ALLOWED_TRANSITIONS` (`02-workflow-state-machine.md`).
- Same-status = no-op.
- On success: stamps `started_at` / `completed_at` per lifecycle rules, appends `status_changed` event with `{old_status, new_status, changed_by}`.

Error summary

Code	Detail	When
401	`missing_auth_header`	`X-User-Email` missing or empty.
401	`unknown_user`	Email not in `users`.
403	`cross_org_access_forbidden`	Body/query asserts a different org or location.
400	`invalid_status`	Bad status string.
400	`invalid_transition: a -> b…`	Disallowed state-machine edge.
400	`invalid_initial_status: …`	Creating with status other than scheduled/in_progress.
400	`location_not_found`	`location_id` does not exist.
404	`encounter_not_found`	Missing or cross-org encounter.

Data Model

SQLite (local dev). Schema reproduced by Alembic migrations

`43ccbf363a8f → a1b2c3d4e5f6`. No schema changes in this auth phase.

Tables

`organizations`

column	type	constraints
id	INTEGER	PK
name	VARCHAR(255)	NOT NULL
slug	VARCHAR(255)	NOT NULL, UNIQUE
created_at	DATETIME	NOT NULL, default `now()`

`locations`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
name	VARCHAR	NOT NULL
created_at	DATETIME	NOT NULL, default `now()`

`users`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
email	VARCHAR	NOT NULL, UNIQUE
full_name	VARCHAR	NULL

column	type	constraints
role	VARCHAR	NOT NULL, default `"admin"`
created_at	DATETIME	NOT NULL, default `now()`

`users.email` is the authentication key for the dev auth layer.

`users.organization_id` is the authoritative source of caller org context.

`encounters`

Unchanged. Indexed on `organization_id`, `location_id`, `patient_identifier`, `status`.

`workflow_events`

Unchanged. Indexed on `encounter_id`, `event_type`.

Relationships

```
organizations 1─┬─* locations
                ├─* users          (identity source for dev auth)
                └─* encounters ─* workflow_events
locations     1─* encounters
```

See `docs/diagrams/er-diagram.md`.

Seeded tenants (two, for scoping proof)

Org 1 — `demo-eye-clinic` (id=1)

- Location: `Main Clinic` (id=1)
- Admin: `admin@chartnav.local` / id=1

encounter_id	patient	provider	status	events
1	PT-1001	Dr. Carter	`in_progress`	3
2	PT-1002	Dr. Patel	`review_needed`	5

Org 2 — `northside-retina` (id=2)

- Location: `Northside HQ` (id=2)

- Admin: `admin@northside.local` / id=2

encounter_id	patient	provider	status	events
3	PT-2001	Dr. Ahmed	`scheduled`	1

Event taxonomy

event_type	When	Shape
`encounter_created`	On `POST /encounters` / seed	`{"status":"...", "created_by":"<email>"}`
`status_changed`	Every successful transition	`{"old_status":"...", "new_status":"...", "changed_by":"<email>"}`
user-supplied	`POST /encounters/{id}/events`	Any JSON

05 — Build log

Build Log

Reverse-chronological. Each entry is a concrete, verifiable step.

2026-04-17 — Phase: dev auth + org scoping

Step 1 — Baseline

- Starting head commit: `505f025` (strict state machine + filtering phase).
- Alembic head still `a1b2c3d4e5f6` — no schema changes needed this phase.

Step 2 — Auth foundation

- New `apps/api/app/auth.py` : `Caller` dataclass, `require_caller` FastAPI dependency, `ensure_same_org` guard.
- `require_caller` reads `X-User-Email` , looks up the user by email, and returns structured context. 401 on missing/empty header; 401 on unknown email.
- Added `GET /me` route that echoes the resolved caller — proves the auth wiring end-to-end.

Step 3 — Org scoping on encounter routes

- `apps/api/app/api/routes.py` :
- `GET /encounters` now always filters `organization_id = caller.organization_id` . `?organization_id=` query becomes a lens: mismatch → 403.
- `GET /encounters/{id}` , `GET /encounters/{id}/events` , `POST /encounters/{id}/events` , `POST /encounters/{id}/status` use a shared `_load_encounter_for_caller` helper that returns 404 on cross-org (no existence oracle).
- `POST /encounters` forces `organization_id` to the caller's org via `ensure_same_org` ; location must also belong to caller org or 403.
- `encounter_created` and `status_changed` events now record the acting user email (`created_by` / `changed_by`).

Step 4 — Seed expansion

- `scripts_seed.py` now seeds two organizations (demo-eye-clinic + northside-retina) with their own admins, locations, and encounters — enough to actually exercise scoping.

- Still idempotent across N runs; verified with back-to-back executions.

Step 5 — Local verification

- Fixed a bug introduced during auth scaffolding: `auth.py` initially resolved `chartnav.db` from the wrong parent (`apps/api/app/chartnav.db`); corrected to `Path(__file__).resolve().parents[1] / "chartnav.db"` (= `apps/api/chartnav.db`). Before fix all scoped routes returned 500.
- Post-fix: full matrix green — see `06-known-gaps.md` for the proof table.

Step 6 — Documentation

- Updated `01-current-state.md` , `03-api-endpoints.md` , `04-data-model.md` , `05-build-log.md` , `06-known-gaps.md` .
- Added `07-auth-and-scoping.md` .
- Updated diagrams: `system-architecture.md` , `api-data-flow.md` , `er-diagram.md` .
- Regenerated `docs/final/chartnav-workflow-state-machine-build.{html,pdf}` so the consolidated artifact now covers Phase 1 (workflow spine) + Phase 2 (state machine + filtering) + Phase 3 (auth + scoping).

Step 7 — Git hygiene

- DB reset to pristine seeded state before commit.
- SQLite `.db` file stays gitignored.
- One atomic commit with code + docs + regenerated final artifacts.

2026-04-17 — Phase: workflow state machine + filtering (prior)

See original entry preserved below for history.

Step 2 — Strict status state machine

- Added `ALLOWED_TRANSITIONS` map.
- Edges: forward `scheduled→in_progress→draft_ready→review_needed→completed` ; rework `draft_ready→in_progress` , `review_needed→draft_ready` .
- All other edges → 400 `invalid_transition` with explicit allowed-next-states listing.
- Same-status = no-op.
- `in_progress` stamps `started_at` if null; `completed` stamps `completed_at` (and backfills `started_at` if null).

Step 3 — Filters

- `GET /encounters` accepts `organization_id`, `location_id`, `status`, `provider_name`. AND-ed, parameterized.

Step 4 — Seed coverage

- Two encounters covering `in_progress` and `review_needed` with realistic event history.

Step 5 — Verification

- Filter matrix + state-machine matrix all green.

2026-04-17 — Phase: workflow spine (earlier)

Migration `a1b2c3d4e5f6`

- Added `encounters` and `workflow_events` tables with FKs and indexes.

Seed

- Idempotent demo org / location / admin / encounter / events.

Initial routes

- 6 encounter endpoints (3 read, 3 write).

06 — Known gaps + verification matrix

Known Gaps & Verification Matrix

Verification matrix — dev auth + org scoping phase

All checks executed against local `uvicorn` with a fresh seeded DB.

Auth

Case	Expected	Actual
`GET /me` no header	401	401
`GET /me` unknown email	401	401
`GET /me` empty header	401	401
`GET /me` org1 admin	200 + `organization_id=1`	200 + `organization_id=1`
`GET /me` org2 admin	200 + `organization_id=2`	200 + `organization_id=2`

Encounter listing

Case	Expected	Actual
`GET /encounters` no auth	401	401
Org1 `GET /encounters`	`[1, 2]`	`[1, 2]`
Org2 `GET /encounters`	`[3]`	`[3]`
Org1 `GET /encounters?organization_id=2` (cross-org lens)	403	403
Org1 `GET /encounters?organization_id=1` (own lens)	`[1, 2]`	`[1, 2]`

Encounter read

Case	Expected	Actual
Org1 `GET /encounters/1`	200	200
Org1 `GET /encounters/3` (other org)	404	404
Org2 `GET /encounters/3`	200	200

Case	Expected	Actual
Org2 `GET /encounters/1` (other org)	404	404
Org1 `GET /encounters/1/events`	200	200
Org1 `GET /encounters/3/events` (cross)	404	404

Encounter create

Case	Expected	Actual
Org1 POST own-org body	201	201
Org1 POST body `organization_id=2` (cross)	403	403
Org1 POST own org but `location_id` belongs to other org	403	403

Encounter mutate

Case	Expected	Actual
Org1 POST `/encounters/1/events` (own)	201	201
Org1 POST `/encounters/3/events` (cross)	404	404
Org1 POST `/encounters/1/status` `draft_ready` (valid)	200	200
Org2 POST `/encounters/1/status` (cross)	404	404
Org1 POST `/encounters/1/status` `completed` (invalid transition)	400	400

Real gaps (prioritized for next phase)

1. **Dev-only auth** — `X-User-Email` is trivially spoofable. Must graduate to JWT or an identity provider before any hosted deployment. The abstraction point (`require_caller`) is already in place.

2. **No role-based authorization** — every seeded user is `admin` . Need `role` -gated actions (clinician vs. admin vs. reviewer) for the state machine edges.

3. **`/organizations` , `/locations` , `/users` still unscoped** — left open intentionally this phase. Next auth phase should either scope them to caller org or gate by role.

4. **No pagination / cursor** on `GET /encounters` .

5. **No encounter update / delete / cancel** path; no `cancelled` status.

6. **No automated tests** — verification is still manual curl; need pytest + httpx `TestClient` covering auth, scoping, state machine, filters.

7. **No ORM / sessions** — raw `sqlite3` with per-request connections. Works for SQLite; Postgres parity untested.

8. **CORS `allow_origins=["*"]` ** — must tighten before any hosted deploy.

9. **No audit log distinct from workflow_events** — auth failures, scoping violations are not persisted.

10. **No rate limiting**.

Auth & Org Scoping

Auth model (dev)

This is a **local/dev** auth layer, not production identity. There are no

passwords, tokens, signatures, or sessions. The contract:

1. Every protected request must include header `X-User-Email: <email>`.

2. The server looks up that user in the `users` table.

3. The caller's `organization_id` is derived from that row.

4. The caller context (`Caller` dataclass) is attached to the request via

FastAPI's `Depends(require_caller)`.

**No body or query parameter can override the caller's resolved

`organization_id`.** Attempts to do so are rejected with 403.

Source

- Dependency: `apps/api/app/auth.py` — `require_caller`, `ensure_same_org`, `Caller`.
- Applied in: `apps/api/app/api/routes.py` on every encounter route and `/me`.

Responses

Situation	HTTP	Detail
Missing / empty `X-User-Email`	401	`missing_auth_header`
Unknown email	401	`unknown_user`
Caller's org ≠ target org (body/query/loc)	403	`cross_org_access_forbidden`
Caller tries to read another org's encounter	404	`encounter_not_found`

Scoping model

Route	Scoping
`GET /encounters`	WHERE `organization_id = caller.organization_id` . `?organization_id=` param is a lens: if it mismatches the caller → 403. `?location_id=` , `?status=` , `?provider_name=` narrow further within the caller's org.
`GET /encounters/{id}`	Same row + cross-org check; mismatched → 404.
`GET /encounters/{id}/events`	Parent-encounter scoped; cross-org → 404.
`POST /encounters`	`organization_id` in body must equal caller org → 403 otherwise. Location must also belong to caller org → 403 otherwise.
`POST /encounters/{id}/events`	404 if cross-org.
`POST /encounters/{id}/status`	404 if cross-org. Strict state machine applies after scope check.
`GET /me`	Auth-only; echoes resolved caller.
`GET /organizations` , `/locations` , `/users`	**NOT scoped** this phase — intentional, documented.

404-vs-403 rule

For encounter reads/mutates, cross-org access returns **404** (same shape as "really doesn't exist") so attackers cannot distinguish "exists in another org" from "doesn't exist at all." For explicit body/query assertions that contradict the caller (e.g. `POST /encounters` body `organization_id=2` while caller is org 1), the server returns **403** because the intent is unambiguous and should fail loudly.

Illustrated flow

What this phase explicitly does NOT do

- No password auth, no JWT, no session cookies.
- No role-based access control (admin vs. clinician etc.) — every seeded

user currently has `role = "admin"` .

- No rate limiting or audit log beyond the `workflow_events` breadcrumb.
- `/organizations` , `/locations` , `/users` remain unauthenticated.

These are the right handoff points for the *next* auth phase.

Diagram — System architecture

System Architecture

```
flowchart LR Client["Client
(curl / apps/web)"] FastAPI["FastAPI
apps/api/app/main.py"] Auth["require_caller
apps/api/app/auth.py"] Router["APIRouter
apps/api/app/api/routes.py"] SM["State Machine
ALLOWED_TRANSITIONS"] Scope["Org Scoping
ensure_same_org"] DB[("SQLite
apps/api/chartnav.db")] Alembic["Alembic
apps/api/alembic/"] Seed["scripts_seed.py
(2 orgs)"] Client -->|"HTTP + X-User-Email"| FastAPI FastAPI --> Auth Auth -->|"lookup by email"|
DB Auth -->|"Caller ctx"| Router Router --> Scope Router --> SM Router -->|"scoped SQL: WHERE
organization_id = caller.org"| DB Alembic -.->|"upgrade head"| DB Seed -.->|"idempotent seed"| DB
```

Diagram — Encounter status machine

Encounter Status State Machine

stateDiagram-v2 [*] --> scheduled [*] --> in_progress : walk-in scheduled --> in_progress in_progress --> draft_ready draft_ready --> review_needed draft_ready --> in_progress : rework review_needed --> completed review_needed --> draft_ready : kick back completed --> [*]

Legend: any edge not shown is rejected at `POST /encounters/{id}/status`

with HTTP 400 `invalid_transition`.

Diagram — ER

ER Diagram

erDiagram organizations ||--o{ locations : "has" organizations ||--o{ users : "employs (identity + org lens)" organizations ||--o{ encounters : "owns" locations ||--o{ encounters : "hosts" encounters ||--o{ workflow_events : "emits" organizations { int id PK string name string slug UK datetime created_at } locations { int id PK int organization_id FK string name datetime created_at } users { int id PK int organization_id FK string email UK string full_name string role datetime created_at } encounters { int id PK int organization_id FK int location_id FK string patient_identifier string patient_name string provider_name string status datetime scheduled_at datetime started_at datetime completed_at datetime created_at } workflow_events { int id PK int encounter_id FK string event_type text event_data datetime created_at }

Dev auth note

`users.email` is the authentication key consumed from the `X-User-Email`

header. `users.organization_id` is the authoritative source of the

caller's organization scope — never derived from body or query params.

Seeded tenants

org_id	slug	admin email	location_id	encounter_ids
1	`demo-eye-clinic`	admin@chartnav.local	1	1, 2
2	`northside-retina`	admin@northside.local	2	3

Diagram — API / data flow

API / Data Flow

sequenceDiagram autonumber participant C as Client participant Auth as require_caller participant R as Route handler participant SM as State Machine participant DB as SQLite Note over C,DB: Happy path — org1 advances own encounter C->>Auth: POST /encounters/1/status {"status":"draft_ready"}
X-User-Email: admin@chartnav.local Auth->>DB: SELECT user WHERE email=? DB-->>Auth: {user_id, organization_id=1, ...} Auth-->>R: Caller(org=1) R->>DB: SELECT encounter 1 ... WHERE id=? DB-->>R: row(org=1) R->>R: encounter.organization_id == caller.org ✓ R->>SM: is (in_progress → draft_ready) allowed? SM-->>R: yes R->>DB: UPDATE + INSERT status_changed{changed_by=caller.email} R-->>C: 200 updated Note over C,DB: Cross-org read — returns 404, not 403 C->>Auth: GET /encounters/3
X-User-Email: admin@chartnav.local Auth-->>R: Caller(org=1) R->>DB: SELECT encounter 3 DB-->>R: row(org=2) R->>R: encounter.organization_id != caller.org R-->>C: 404 encounter_not_found Note over C,DB: Cross-org write assertion — returns 403 C->>Auth: POST /encounters {"organization_id":2,...}
X-User-Email: admin@chartnav.local Auth-->>R: Caller(org=1) R->>R: body.organization_id != caller.org R-->>C: 403 cross_org_access_forbidden Note over C,DB: Missing / unknown identity C->>Auth: GET /encounters Auth-->>C: 401 missing_auth_header C->>Auth: GET /encounters
X-User-Email: ghost@nowhere Auth->>DB: SELECT user DB-->>Auth: ∅ Auth-->>C: 401 unknown_user